Runtime-Tracking Coverage-Based Test Case Prioritization for Effective Regression Testing in Java Systems

Alessandro Massimo Mermali†, Bianca Maria Cerino†,
Guglielmo Giovanni Grifoni†

School of Science and Technology, University of Camerino, Camerino, 62032, Italy.

Contributing authors: alessandroma.mermali@studenti.unicam.it;
biancamaria.cerino@studenti.unicam.it; guglielmogio.grifoni@studenti.unicam.it;
†These authors contributed equally to this work.

Abstract

Regression testing is a critical but resource-intensive activity in modern software development, particularly in object-oriented systems where abstraction, inheritance, and dynamic behavior complicate accurate impact analysis. Traditional test case prioritization techniques often rely on static analysis or coarse-grained coverage metrics, which can lead to imprecise estimations of change impact and suboptimal fault detection rates. This paper proposes a novel change-aware, coverage-based test case prioritization approach tailored for Java systems. The approach extracts the methods affected by a change between two Git revisions and correlates them with fine-grained, per-test runtime coverage collected via JaCoCo, constructing a weighted graph linking methods and tests. Test cases are then prioritized according to the weights provided by the graph, favoring tests more likely to exercise recently modified logic. By relying on runtime coverage rather than static call-graph approximation, the approach avoids the imprecision that polymorphism and inheritance hierarchies typically introduce in static impact analysis.

Keywords: Testing Prioritization, Coverage, Abstraction, Heritability, Regression Testing, Java Analysis, Object-Oriented Programming, Runtime Tracking

1 Introduction

Regression testing is the practice of re-executing previously passing test cases after a system has changed, to confirm that no new faults, or regressions, have been introduced. It is one of the most consistently applied quality assurance activities in modern software development, precisely because change is constant: every commit, refactor, or bug fix is a candidate source of unintended breakage elsewhere in the system.. The problem is compounded by the short release cycles and continuous integration (CI) practices that dominate contemporary development, where the delay between a code change and test feedback is expected to be minimal, yet the volume of tests to run keeps increasing [1].

Test Case Prioritization (TCP) addresses this cost by reordering, rather than reducing, the execution of a test suite, scheduling the test cases most likely to reveal a fault as early as possible [2, 3]. Because it does not discard any test, TCP is comparatively low-risk to adopt relative to test selection or suite reduction, and it has consequently attracted a large and continuously

growing body of research: two independent systematic literature reviews published within the last three years each catalog dozens of proposed TCP methods, spanning coverage-based, history-based, cost-based, requirements-based, and hybrid approaches [1, 2].

A significant obstacle to the precision of these methods, however, is object-oriented design itself. Principles such as encapsulation, inheritance, and polymorphism are central to how modern Java systems are structured [4], but they are also precisely what undermines static approximations of program behavior: polymorphic method calls and inheritance hierarchies mean that the method actually executed at runtime frequently cannot be determined by inspecting the call site alone. Impact analysis techniques that rely on static call graphs therefore risk both under- and over-approximating which parts of a system are affected by a given change, a limitation that becomes more pronounced as class hierarchies deepen [5].

This paper makes the following contributions:

1. A git-integrated implementation of a change-aware TCP technique for Java, combining a statically built method-level call graph, with per-test JaCoCo coverage to prioritize tests by their association with a given change.
2. An evaluation of the approach on a real-world Java project, assessing its effectiveness at regression TCP compared to coarser-grained baselines.
3. A complementary dataset composed of graph representations and test priority queues used in the training phase.

To evaluate the effectiveness of the proposed approach, this work investigates the following research question:
RQ: Does a change-impact graph combining Git-derived method-level changes with runtime test coverage improve regression test case prioritization in Java systems?
By addressing this question, we aim to evaluate whether prioritizing tests according to this graph yields earlier fault detection than a baseline ordering.

The remainder of this paper is organized as follows: Section 2 provides an overview of the key concepts underlying this work. Section 3 reviews the existing literature on regression testing and TCP. Section 4 describes the methodology behind our approach. Section 5 provides the specifics of the implementation. Section 6 reports and discusses the results of this evaluation. Section 7 discusses the findings and limitations. Finally, Section 8 concludes the paper and discusses future work.

2 Background

This section introduces the fundamental concepts underlying the proposed approach. It provides a brief overview of regression testing and TCP, followed by the object-oriented characteristics of Java systems that influence change impact analysis (CIA). These concepts establish the foundation for the methodology presented in Section 4.

2.1 Regression Testing

Regression testing is the activity of re-executing a system's existing test cases after the system under test has been modified, in order to verify that the change has not introduced new failures, or regressions, into previously working functionality [1, 6].

In its most conservative form, regression testing follows a retest-all strategy, in which the entire existing test suite is re-executed after every modification. While straightforward, this approach scales poorly: as a project matures, its test suite tends to grow substantially, and re-running it in full after each change becomes increasingly impractical, particularly under the short release cycles and CI practices common in modern development, where fast feedback is expected on every commit [1]. This tension between test suite size and available time budget motivates several regression testing techniques. These include test case selection (TCS), which executes only tests relevant to a change; test suite reduction (TSR), which permanently removes redundant tests; test suite augmentation (TSA), which generates additional tests; and TCP, which reorders test execution without discarding any test.[1, 6].

Modern regression testing techniques increasingly exploit information about the changes introduced between software revisions. By identifying the program entities affected by a modification, change-aware approaches can focus testing effort

on the portions of the system most likely to contain regression faults.

2.2 Test Case Prioritization

TCP seeks to order test cases so that those with the highest likelihood of revealing faults are executed earlier. Unlike TCS or TSR, TCP preserves the complete test suite while improving the rate of early fault detection, making it one of the most extensively studied techniques in regression testing [2].

TCP techniques can be classified according to the information they exploit, such as code coverage, historical execution data, software requirements, or execution cost. Among these, coverage-based approaches are the most widely studied and adopted in practice [2, 7]. Another common distinction separates *static* techniques, which rely solely on source-code information, from *dynamic* techniques, which exploit runtime information such as code coverage collected during test execution [8]. This distinction is particularly relevant for object-oriented systems, where runtime behavior may differ significantly from static approximations. More recently, a third line of work has framed prioritization itself as a learning problem rather than a fixed heuristic, using Machine Learning (ML) techniques such as Reinforcement Learning (RL) [3].

The effectiveness of TCP techniques is commonly evaluated using the Average Percentage of Faults Detected (APFD), which measures how rapidly faults are exposed during test execution [2]. Although alternative evaluation metrics have been proposed, APFD remains the de facto standard for comparing prioritization strategies due to its simplicity and widespread adoption [1, 8].

2.3 OOP Characteristics Affecting Testing

Object-oriented programming (OOP) organizes software around objects that encapsulate data and behavior, with encapsulation, inheritance, and polymorphism among its foundational principles. Encapsulation restricts direct access to an object's internal state, exposing only a defined interface of methods; inheritance allows a subclass to acquire and extend the properties and behavior of a superclass, promoting reuse; and polymorphism allows an object of a subclass to be treated as an instance of its parent type, enabling a single piece of calling code to operate over multiple concrete implementations [4].

While these principles are central to the modularity and reusability benefits of OOP, they also introduce specific challenges for static impact analysis, and by extension for TCP techniques that rely on it. Because polymorphism allows the method actually invoked at a given call site to vary depending on the runtime type of the receiving object, a static analysis of the source code alone often cannot determine with certainty which method implementation will execute. As a consequence, techniques that approximate change impact through static call graphs risk both missing affected methods and including methods that are actually unaffected. Runtime-derived information does not suffer from this ambiguity, since it reflects the methods that were genuinely exercised for a given input, regardless of how many levels of inheritance or polymorphic dispatch were involved in reaching them. This distinction motivates the coverage-based, runtime-driven design of the approach proposed in this paper.

3 Literature Review

Over the years, TCP has received significant research attention in the context of regression testing and CI, with the aim of accelerating fault detection by reordering the execution of tests. This section provides an overview of the evolution of TCP, along with some of its Java-based implementations, establishing the background that motivates the approach presented in this paper.

3.1 Coverage Test Prioritization

The seminal work by Rothermel et al. [6] formalized the problem of TCP and showed that coverage information can be exploited to improve the rate of fault detection. This establishes the basic intuition behind coverage-based TCP: test cases should not be treated uniformly, because different tests exercise different parts of the system and therefore provide different levels of regression-testing value.

However, traditional coverage-based TCP techniques are often limited by the granularity and type of information they use. If prioritization is based only on coarse-grained coverage, it may fail to identify which tests are most relevant to a specific change. This limitation is particularly important in Java systems. For this reason, method-level coverage provides a more precise basis for change-aware prioritization.

CIA provides an important conceptual foundation for this kind of change-aware regression testing. Ryder and Tip [5] describe CIA for object-oriented programs as a way to identify program entities and test drivers that may be affected by a modification. Similarly, Harrold et al. [9] investigate regression test selection for Java software, emphasizing the need for techniques that account for language-specific characteristics. These contributions show that regression testing in Java cannot rely only on superficial file-level changes, because inheritance, overriding, interface calls and other OOP-specific dependencies can complicate the relationship between a source-code modification and the tests affected by it.

A concrete realization of CIA for Java is provided by Chianti [10]. Chianti compares two versions of a Java program, decomposes the differences into atomic changes and identifies tests that may be affected by those changes. Similarly, FaultTracer [11] uses change impact and regression fault analysis to reason about evolving Java programs and to relate changes to potential regression faults. These approaches demonstrate the value of incorporating change information into regression testing.

3.2 Graph-Based Approaches

Several works have adapted regression test prioritization more directly to object-oriented programs by using graph-based representations. Panigrahi and Mall [12] propose a technique for prioritizing regression test cases of object-oriented programs by constructing an intermediate graph representation from the source code and updating it after program changes. Their model considers dependencies relevant to object-oriented systems, including inheritance, aggregation, association, control dependencies and data dependencies. Panda et al.[13] further connect object-oriented CIA and regression test prioritization through an affected slice graph, using affected program slices and coupling-related information to prioritize regression test cases. These approaches show that graph representations can support more informed prioritization by making relationships among program entities explicit.

The distinction between static and dynamic information is central to this problem. Luo et al. [8] provide a large-scale empirical comparison of static and dynamic TCP techniques, showing that different sources of information can lead to different prioritization effectiveness. Static techniques can approximate potential dependencies without executing tests. Dynamic techniques, by contrast, use execution information such as coverage and therefore reflect what actually happened during a test run.

Helm et al. [14] further show that the quality of static call graphs along with their precision and recall can vary substantially depending on the analysis strategy and language features. This observation is especially important for Java, where inheritance, interfaces, and polymorphism can introduce uncertainty into static call-graph construction.

Recent work has also explored learning-based regression test optimization, showing that historical execution data can complement traditional program analysis techniques [15].

3.3 Research Gap

Overall, existing work provides several building blocks for change-aware regression test prioritization. Coverage-based TCP demonstrates that execution information can guide test ordering [6]; Java-specific regression test selection and CIA show that object-oriented language features must be considered explicitly [5, 9, 10]; graph-based and slice-based approaches suggest that relationships among program entities can support more precise prioritization [12, 13]; empirical comparisons of static and dynamic techniques motivate the use of runtime execution data [8]; and studies on static call graphs highlight the risk of relying only on static approximations in Java systems [14].

The gap addressed by this paper lies at the intersection of these directions. Existing techniques often either rely on coarse-grained coverage, focus mainly on affected-test selection, or depend on static dependency information that

may be imprecise in the presence of inheritance and polymorphism.

4 Methodology

This section presents the proposed change-aware TCP approach. The methodology combines Git-based change analysis with fine-grained runtime coverage to identify the tests most closely related to recent code modifications. The approach is organized as a sequence of stages: first, the methods affected by a software revision are extracted from the Git history; next, per-test runtime coverage is collected using JaCoCo; this information is then integrated into a weighted graph that captures the relationship between methods and tests; finally, the resulting graph is used to prioritize the execution order of the test suite.

4.1 TCP Change Aware

Change-Aware TCP is a test prioritization technique that allows tests to be ranked based on the changes introduced in the code. This allows tests with the highest probability of identifying errors caused by changes to be executed first, thus reducing the cost of regression testing.

This technique's approaches can be implemented using different types of metrics, including coverage-based metrics.

In our project, we use a change-aware prioritization strategy by combining three main aspects:

- the identification of methods modified between two versions of the source code;
- the extraction of the set of methods executed by each test;
- the analysis of the impact of code changes on the propagation of potential faults across the system.

All of this helps us run fault-revealing tests earlier, enabling a fail-fast testing strategy.

4.2 Per-Test Coverage

Per-test coverage is a software testing metric that tracks which parts of the code are executed by each individual test, rather than considering the entire test suite as a single entity.

The main difference between traditional code coverage and per-test coverage is that the former provides an overall view of the coverage achieved by the entire test suite, while the latter identifies which parts of the code are covered by each specific test.

This is an important property in our project because it allows us to identify which tests cover the modified methods in the new version of the code, instead of only knowing that the test suite, as a whole, covers those modified methods.

4.3 Weighted Graph Definition

The information extracted from static analysis, change analysis and runtime coverage is integrated into a directed weighted graph. The graph is defined at method-level granularity, so that both main methods and test methods can be represented as first-class nodes.

We defined the software as a graph S

$$S = (M, E)$$

Each node $m \in M$ represents a method-like program element, including structures typical of OOP languages such as constructors and interface methods. A node is identified by a stable method signature containing the declaring class, method name, parameter types and return type. In addition, each node stores metadata such as its own source file, line range, visibility and method kind. Nodes are also associated with an initial risk value.

Each edge $e \in E$ represent risk-propagation relationships between methods. The graph contains three categories of edges:

- *Call-impact edges* connect a method to the method that invokes it. These edges are obtained through static source analysis and are further characterized by the kind of call.
- *Contract-impact edges* connect abstract or interface-level contract methods to concrete implementations. These edges are used to model the effect of changed that may propagate through inheritance and overriding relationships.
- *Test-impact edges* connect production methods to test methods that cover them at runtime. These edges represent direct evidence that a given tests uses a given method.

Each edge is assigned a weight that determines how strongly risk propagates through that relation. Instead of assigning a separate parameter to every individual edge, the model learns shared weights for relation types.

4.3.1 Initial Risk Assignment

Given the previously explicited definition of the graph, let C be the subset of M containing the methods that have been changed and $c \in C$ a specific changed method.

The initial risk function r for c is defined as

$$r_0(c) = (w_{CE} \cdot CE(c) + w_{chE} \cdot chE(c) + w_{cs} \cdot CS(c))$$

where CE is the `callExposure`, a metric that measures how many call-impact edges depend on a method; chE is the `changeExtent`, how much of a method was modified; CS is the `changeSensitivity`, an estimation of how structurally important the changed lines are. All these values, as well as the risk itself, are normalized in $[0, 1]$. w_{CE}, w_{chE}, w_{CS} are weights that measure how important that specific feature is when computing the risk.

$$w_{CE} + w_{chE} + w_{CS} = 1$$

The initial risk definition ensures that the risk source is local and explicit: before any propagation step, only the changed method contributes regression risk to the graph.

4.3.2 Prioritization Strategy

The prioritization strategy is based on risk propagation through the graph. The risk initially assigned by the graph is then propagated through the weighted graph along outgoing edges.

Let C be the set of changed methods and let $r_0(c)$ be the initial risk assigned to a changed method $c \in C$. For a path

$$p = (v_0, v_1, \ldots, v_k)$$

starting from a changed method $v_0 \in C$, the contribution propagated along that path is computed as

$$r(p) = r_0(v_0) \cdot \prod_{i_0}^{k-1} w(v_i, v_{i+1})$$

where $w(v_i, v_{i+1})$ is the weight of the edge between two consecutive nodes.

The final risk score of a node is the sum of all bounded path contributions reaching that node:

$$R(v) = \sum_{p \to v} r(p)$$

After propagation, only test-method nodes are considered for prioritization. Tests are ranked in descending order of their propagated risk score. Ties are resolved deterministically using the method identifier.

This means that a test is prioritized not merely because it belongs to a changed file or class, but because risk can flow from changed methods to said test through a sequence of weighted semantic relationships.

4.3.3 Weight Tuning

Instead of heuristic pre-defined edge weights, we decided to apply ML techniques to derive them from historic cases.

Given an input composed of tuples of multiple-graphs and expected priority queue, our model compares the predicted ranking with the expected priority queue. At the first mismatch, let t^+ be the expected test and (t^-) the incorrectly predicted test. For each edge group g, the trainer computes the normalized path support difference

$$\Delta_g = \sum_{e:g(e)=g} \left(\widehat{S}_e(t^+) - \widehat{S}_e(t^-) \right)$$

where $\widehat{S}_e(t)$ is the normalized contribution of edge e among all paths reaching test t. The corresponding shared weight is then updated as

$$w \leftarrow (w + \eta \Delta_g)$$

where η is the `learning rate`.

Thus, weights that contribute more to the expected test are increased, while weights that mainly support the wrongly predicted test are decreased.

5 Implementation

In this section we discuss the implementation[1] choices of the model.

5.1 Git-Based Change Analysis

The first stage of the implementation identifies, at method-level precision, which parts of a Java codebase were altered by a given change, rather than treating a change as a property of a whole file.

Given a repository and two revisions, namely a *base* and a *head*, the file-level diff between them is computed using ***JGit***, restricted to `.java` files, with rename detection applied so relocated files are not mistaken for new ones. For each changed file, the diff yields two sets of line numbers: those modified in the *base* revision and those modified in the *head* revision.

Since a test suite operates on methods rather than raw line numbers, the source file is retrieved and parsed independently at each of the two revisions using ***JavaParser***, extracting every method's, constructor's, and compact constructor's enclosing class, parameter types, and line range. Intersecting the diff's changed lines with these method ranges identifies which methods were altered on each side of the change.

This is done separately for the *base* and *head* revisions, since a change can manifest differently on each side. The two resulting sets of changed methods are then merged into a single, deduplicated collection, with the specific changed line numbers preserved for each method.

The output of this stage is therefore not a diff in the conventional sense, but a precise, method-addressable description of change: a set of uniquely identified methods, each annotated with the lines actually altered within it.

5.2 JaCoCo

JaCoCo (Java Code Coverage) is an open-source library that measures which parts of a Java program are actually executed during a run, and is the tool used in this work to obtain per-test method coverage.

It works by attaching a Java agent to the JVM via the `-javaagent` flag; as classes are loaded, the agent instruments their bytecode in memory, inserting *probes* that record whether a given method, line, or branch is executed. No source code or build file needs to be modified, and the original `.class` files remain untouched. As the instrumented program runs, each executed *probe* accumulates in an in-memory execution data structure held by the agent.

By default, JaCoCo writes this data to a `.exec` file once the JVM terminates, producing a single aggregate coverage record for the entire run. This work instead uses JaCoCo's runtime API to read and reset the accumulated data on demand, while the JVM is still running. Querying and resetting the data immediately after each test method finishes makes it possible to isolate that test's coverage contribution within a single JVM lifetime, so that one build execution of the whole test suite yields one coverage record per test, rather than one record for the suite as a whole.

Once retrieved, execution data is related back to the original class files through JaCoCo's analysis API, which determines, for every class, which methods, lines, and branches were exercised. Because this information reflects actual runtime behavior rather than a static approximation, it correctly captures which concrete method implementation was invoked at each call site, including under inheritance and polymorphism.

5.3 Weighted Graph Construction

The information provided by the static analysis and by the JaCoCo reports are then used to build the weighted graph previously defined in Section 4.3.

5.3.1 Dataset and Training

The edge weights used by the prioritization strategy are tuned offline using mutation testing. Mutation testing is not part of the prioritization procedure itself, rather it is used to construct training examples that indicate which tests are capable of detecting faults introduced into specific methods.

[1] Implementation available at:
https://github.com/ammermali/TestCasePrioritization

Therefore, we created a specific dataset[2], based on the PredictiveRTS [15] and PIT [16] tools. These tools provide two key pieces of information relatively to each subject: a mutated method and the set of killing tests for said method. The mutated method is mapped to the corresponding method node in the graph, while the killing tests are normalized and mapped to test-method nodes.

Each valid mutation produces a training case. In that case, the graph is copied and the mutated method is marked as the only initially risky node. The expected priority queue is derived from the killing tests reported by PIT. Mutations are discarded in certain situation if they may impair the training.

The resulting dataset is shown in Table 1.

The resulting model is therefore a compact set of semantic edge weights. These weights can be reused on new revisions without requiring PIT, mutation reports or access to the original training projects. For the sake of the training we defined the `repetitions` parameter. This parameter defines how many iterations the training goes through. If the dataset is composed by n instances, the overall training goes through $\frac{repetitions}{n}$ epochs. The weight edges are then trained with $\eta = 0.02$ and repetitions $= 1000$.

6 Evaluation

This chapter evaluates the proposed test case prioritization approach. The goal of the evaluation is to measure whether the model is able to rank fault-revealing tests near the top of the test suite when applied to real software defects.

6.1 Evaluation Metrics

We evaluate each produced test ranking by comparing it against the set of triggering tests associated with a real defect. A triggering test is a test that fails on the buggy version of the program and therefore exposes the fault.

Let $R = [t_1, t_2, \ldots, t_n]$ be the ranked list of test methods produced by the model, and let $T_{trigger}$ be the set of triggering tests for a given bug.

The **first trigger rank** measures the position of the first triggering test in the prioritized test list:

$$FirstTriggerRank = \min_{t \in T_{trigger}} rank(t)$$

Lower values are better. A value of 1 means that the model selected a fault-revealing test as the highest-priority test.

Recall@k measures the fraction of triggering tests that appear among the first k prioritized tests:

$$Recall@k = \frac{|T_{trigger} \cap R_{top-k}|}{|T_{trigger}|}$$

We report Recall@1, Recall@3, Recall@5, and Recall@10. Higher values indicate that more fault-revealing tests are placed near the top of the ranking.

Mean Reciprocal Rank (MRR) captures how early the first triggering test appears:

$$MRR = \frac{1}{FirstTriggerRank}$$

For a single evaluation case, MRR is high when the first triggering test is found early. When aggregated across multiple bugs, we report the mean MRR over all successful cases.

We also report the **Average Percentage of Faults Detected (APFD)**, a standard metric in test case prioritization. In this evaluation, APFD is computed over the set of triggering tests associated with each bug:

$$APFD = 1 - \frac{\sum_{i=1}^{m} TF_i}{n \cdot m} + \frac{1}{2n}$$

where n is the total number of ranked tests, m is the number of triggering tests, and TF_i is the rank position of the i-th triggering test. Higher APFD values indicate that fault-revealing tests are detected earlier in the prioritized test order.

6.2 Defects4J Evaluation

We use Defects4J as an external benchmark of real Java defects. Defects4J provides reproducible buggy and fixed program versions, together with metadata such as the list of triggering tests for each bug. This makes it suitable for evaluating whether the prioritization model can identify tests that reveal real faults.

For each selected Defects4J bug, we perform the following procedure:

[2]Dataset available at: https://github.com/ammermali/TestCasePrioritization-Dataset

Table 1 Dataset Composition.

Project	Main	Test	Nodes	Edges	Excluded	Mutations
apache_commons-validator	1,286	11	1,297	1,772	0	2,008
apache_commons-collections	6,709	1,990	8,699	13,661	0	9,763
apache_commons-codec	1,375	1,013	2,388	4,189	2	4,389
google_gson	1,603	1,410	3,013	7,202	4	4,220
jhy_jsoup	2,489	1,560	4,049	11,754	30	1,652
apache_commons-text	1,513	1,294	2,807	5,407	79	784
Total	**15,316**	**7,375**	**22,691**	**44,631**	**120**	**22,881**

1. Check out the buggy program version.
2. Extract the Defects4J metadata, including the triggering tests.
3. Identify the fixed revision corresponding to the bug.
4. Compute the changed methods from the buggy-to-fixed diff.
5. Build the impact graph for the buggy version.
6. Seed the changed methods with initial risk.
7. Propagate risk through the graph.
8. Rank test methods by their propagated risk score.
9. Compare the ranked tests against the Defects4J triggering tests.

For each bug, the evaluation produces the changed methods, the triggering tests, the complete ranked test list and the metrics described in Section 6.1. Failed cases, such as checkout, compilation or parsing failures, are reported separately and excluded from the aggregate metrics.

Table 2 Aggregate Defects4J Evaluation Metrics.

Metric	Value
Recall@1	0.5214
Recall@3	0.5214
Recall@5	0.5393
Recall@10	0.5393
MRR	0.5929
APFD	0.5363

The final results are aggregated across all successful Defects4J cases; the results are shown in Table 2.

7 Discussion

This section interprets the evaluation results, discusses the main limitations of the study and outlines directions for extending the proposed approach. The findings should be considered as evidence of the feasibility and effectiveness of combining change information, static dependency relationships and per-test runtime coverage, rather than as definitive evidence that the approach generalizes to every Java system and development environment.

7.1 Interpretation of Results

The results reported in Table 2 show that the proposed approach achieved almost perfect metrics. In particular, `recall@1` is equal to 0.5214, indicating that approximately 52.14% of the triggering tests were placed at the first position of the priority queue on average. This result suggests that the model is frequently able to assign a high risk score to fault-revealing tests, although its predictions are not consistently correct across all evaluated defects.

The identical value obtained for `Recall@1` and `Recall@3` indicate that extending the considered portion of the ranking from the first test to the first three tests di non retrieve additional triggering tests. Recall increases slightly to 0.5393 at `Recall@5`, showing that a small number of additional fault-revealing tests appeared between the fourth and fifth positions. The absence of further increase suggests that no additional triggering tests were found between position six and ten. Therefore, when the approach identifies a relevant test, it tends to place it very close to the beginning of the ranking; otherwise, the remaining triggering tests may appear considerably later in the queue.

The APFD value of 0.5363 provides an overall measure of how early the triggering tests were

detected across the complete rankings. This value shows that fault-revealing tests were, on average, detected slightly before the midpoint of the test execution order. However, the APFD result alone is insufficient to establish whether the proposed approach outperforms alternative prioritization strategies. A comparison with appropriate baselines, such as the original test order, random ordering, and conventional coverage-based prioritization, is required before claiming a measurable improvement.

Overall, the results provide preliminary evidence that the propagated risk score can identify relationships between changed methods and fault-revealing tests. The relatively high `Recall@1` compared with the later recall values suggests that the model is particularly effective when it assigns a strong risk signal to a triggering test. Nevertheless, the limited improvement between `Recall@1` and `Recall@10` also indicates that the approach may struggle to recover additional triggering tests when they are not assigned the highest priority. These findings motivate further analysis of the cases in which triggering tests receive low scores, as well as a broader evaluation involving additional Defects4J projects and prioritization baselines.

7.2 Limitations and Threats to Validity

A first limitation concerns the external validity of the evaluation. Some of the projects used by Defects4J (evaluation) and PredictiveRTS (training) are in common, which may lead to a data leakage in the training-evaluation loop. Nevertheless, it's important to highlight that the overlapping projects compose less than 15% of the respective sets.

Moreover, the data used for training is limited to the seven projects described in Table 1, future expansion should focus on improve the quality and quantity of this dataset.

Another limitation concerns the methodology used in the computation of the `changeSensitivity` (Section 4.3.1). At the moment, this computation is done through a hard-coded, rule-based assignment. In future expansions, it could be expanded using embedding and similar semantic-based techniques.

Implementation-wise, the current model is limited to JUnit5 and single-module Gradle and Maven projects. Therefore, the results cannot be yet generalized to JUnit4 suites, multi-model repositories, Android applications or distributed systems without further evaluation.

Furthermore, the approach combines runtime coverage with statistically extracted call-impact and contract-impact relationships. Runtime coverage reduces the ambiguity caused by inheritance and polymorphic dispatch for executions that have actually been observed. It does not, however, eliminate all sources of static-analysis imprecision, since indirect risk propagation still depends partly on the completeness and correctness of the statistically constructed graph.

Finally, the current metrics mainly evaluate ranking effectiveness. They do not directly account for test duration, graph-construction cost or the time required to collect per-test coverage. Consequently, the evaluation demonstrate how early triggering tests appear in the ranking, but it does not yet establish the complete cost-benefit trade-off of deploying the technique in a CI pipeline.

8 Conclusion

This paper addresses the research question of wether a change-impact graph combining Git-derived method-level changes with runtime test coverage can improve regression TCP in Java systems. Motivated by the cost of regression testing under continuous integration and the specific imprecision that inheritance and polymorphism introduce into static change-impact analysis, we presented a coverage-based TCP approach built around a statically constructed method-level graph combined with per-test coverage collected via ***JaCoCo***. We implemented this approach as a git-integrated, standalone tool and evaluated it on real-world Java projects from the ***Defects4J*** benchmark to asses its effectiveness.

Several directions remain open for future work. The evaluation was limited to the Defects4J benchmark and to projects using JUnit5; extending it to additional, larger-scale systems, as well as to JUnit4 suites, would strengthen the generality of the results. The current implementation is also restricted to single-module Gradle or Maven projects; supporting multi-module builds

and other build tools would broaden the range od real-world projects the approach can be applied to. Runtime coverage reduces, but does not eliminate, static-analysis imprecision, since indirect risk propagation still depends on the completeness of the statically constructed graph. Finally, the ranking effectiveness does not yet account for test duration, graph-construction cost, or coverage-collection time, leaving the full cost-benefit tradeoff of deploying the technique in a CI pipeline for future work.

References

[1] Greca, R., Miranda, B., Bertolino, A.: State of practical applicability of regression testing research: A live systematic literature review. ACM Comput. Surv. **55**(13s) (2023) https://doi.org/10.1145/3579851

[2] Fernandes, I.P., Martins, L.E.G.: Test case prioritization methods: A systematic literature review. Journal of Software Engineering Research and Development **13**(2), 13–51 (2025) https://doi.org/10.5753/jserd.2025.4504

[3] Bagherzadeh, M., Kahani, N., Briand, L.: Reinforcement learning for test case prioritization. IEEE Transactions on Software Engineering **48**(8), 2836–2856 (2022) https://doi.org/10.1109/TSE.2021.3070549

[4] Sheta, S.: An overview of object-oriented programming (oop) and its impact on software design **24**, 409–419 (2021)

[5] Ryder, B.G., Tip, F.: Change impact analysis for object-oriented programs. PASTE 01, pp. 46–53. Association for Computing Machinery, New York, NY, USA (2001). https://doi.org/10.1145/379605.379661 . https://doi.org/10.1145/379605.379661

[6] Rothermel, G., Untch, R.H., Chu, C., Harrold, M.J.: Prioritizing test cases for regression testing. IEEE Transactions on Software Engineering **27**(10), 929–948 (2001) https://doi.org/10.1109/32.962562

[7] Elbaum, S., Malishevsky, A.G., Rothermel, G.: Test case prioritization: A family of empirical studies. IEEE Trans. Softw. Eng. **28**(2), 159–182 (2002) https://doi.org/10.1109/32.988497

[8] Luo, Q., Moran, K., Poshyvanyk, D.: A large-scale empirical comparison of static and dynamic test case prioritization techniques. CoRR **abs/1801.05917** (2018) 1801.05917

[9] Harrold, M.J., Jones, J.A., Li, T., Liang, D., Orso, A., Pennings, M., Sinha, S., Spoon, S.A., Gujarathi, A.M.: Regression test selection for java software. In: Conference on Object-Oriented Programming Systems, Languages, and Applications (2001). https://api.semanticscholar.org/CorpusID:5323768

[10] Ren, X., Shah, F., Tip, F., Ryder, B.G., Chesley, O.: Chianti: a tool for change impact analysis of java programs. In: Proceedings of the 19th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications. OOPSLA '04, pp. 432–448. Association for Computing Machinery, New York, NY, USA (2004). https://doi.org/10.1145/1028976.1029012 . https://doi.org/10.1145/1028976.1029012

[11] Zhang, L., Kim, M., Khurshid, S.: Faulttracer: a change impact and regression fault analysis tool for evolving java programs. In: Proceedings of the ACM SIGSOFT 20th International Symposium on the Foundations of Software Engineering. FSE '12. Association for Computing Machinery, New York, NY, USA (2012). https://doi.org/10.1145/2393596.2393642 . https://doi.org/10.1145/2393596.2393642

[12] Panigrahi, C.R., Mall, R.: An approach to prioritize regression test cases of object-oriented programs. CSI Transactions on ICT (2013) https://doi.org/10.1007/s40012-013-0011-7

[13] Panda, S., Munjal, D., Mohapatra, D.P.: A slice-based change impact analysis for regression test case prioritization of object-oriented programs. Advances in Software Engineering **2016**(1), 7132404 (2016) https://doi.org/10.1155/2016/7132404

[14] Helm, D., Keidel, S., Kampkötter, A., Düsing, J., Roth, T., Hermann, B., Mezini, M.: Total recall? how good are static call graphs really? In: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis. ISSTA 2024, pp. 112–123. Association for Computing Machinery, New York, NY, USA (2024). https://doi.org/10.1145/3650212.3652114 . https://doi.org/10.1145/3650212.3652114

[15] Zhang, J., Liu, Y., Gligoric, M., Legunsen, O., Shi, A.: Comparing and combining analysis-based and learning-based regression test selection. In: Proceedings of the 3rd ACM/IEEE International Conference on Automation of Software Test. AST '22, pp. 17–28. Association for Computing Machinery, New York, NY, USA (2022). https://doi.org/10.1145/3524481.3527230 . https://doi.org/10.1145/3524481.3527230

[16] Coles, H., Laurent, T., Henard, C., Papadakis, M., Ventresque, A.: Pit: a practical mutation testing tool for java (demo). In: Proceedings of the 25th International Symposium on Software Testing and Analysis. ISSTA 2016, pp. 449–452. Association for Computing Machinery, New York, NY, USA (2016). https://doi.org/10.1145/2931037.2948707 . https://doi.org/10.1145/2931037.2948707